Dynamic Class Scheduler and Countdown

JavaScript Timers and ES6 Modules

THEORY AND CASE STUDY | PRACTICAL 11

Aim

To use JavaScript timers (setTimeout(), setInterval()) and ES6 modules (import/export) to build a dynamic countdown or class scheduler with real-time updates.

Software/Tools Required

- HTML5 for the scheduler form, countdown, and class list.
- CSS3 for responsive layout and clear timer/status states.
- JavaScript ES6+ for timers, modules, and event handling.
- VS Code or any text editor, and Live Server for local development.
- A modern browser with ES-module support; serve the project over HTTP.

Theory

1. setInterval() and live countdowns

setInterval(callback, delay) schedules repeated work. A countdown can calculate the remaining time until a target date and refresh its display once per second. The interval should be cleared when the target is reached or the countdown is replaced.

2. setTimeout() and scheduled alerts

setTimeout(callback, delay) schedules a one-time callback. It is suitable for a reminder or notification after a delay. Timer identifiers allow pending work to be cancelled with clearTimeout().

3. ES6 modules with import and export

ES modules divide a program into files with explicit exports and imports. Separating timer calculations, scheduler data operations, and UI rendering makes each module easier to understand and reuse. Browsers load module scripts using type="module".

4. Countdown and scheduler updates

A scheduler stores class details such as subject, instructor, location, start time, and duration. The application selects the next relevant class, updates the countdown, and refreshes the schedule when a class is added or deleted.

Practical 11 | Case Study and Implementation

Case Study Overview

This case study implements a class scheduler with a live countdown to the next upcoming class. Users enter class details, view a dynamically rendered schedule, and remove entries. Timer, scheduling, and UI logic are separated into ES modules; class data can persist in localStorage between visits.

Key Features Implemented

- Display a live days, hours, minutes, and seconds countdown.
- Refresh the countdown every second with setInterval().
- Schedule a one-time reminder with setTimeout().
- Add and delete classes through an interactive form and list.
- Organize timer, schedule, and rendering code using ES module imports and exports.
- Save and restore scheduled classes using localStorage.

Core Implementation Details

HTML Structure (index.html)

- Provide the class form, upcoming-class summary, countdown container, and schedule list.
- Load the application controller as a module script.

CSS Styling (style.css)

- Use a responsive grid for the scheduler form and schedule.
- Style countdown units, class status, and feedback for readability.

JavaScript Modules

- timer.js exports countdown calculation/start/stop and scheduled-alert helpers.
- scheduler.js exports class loading, saving, add/delete, and next-class selection.
- ui.js exports functions that render countdowns, schedule entries, and notifications.
- main.js imports these helpers, handles form events, and refreshes the UI when class data changes.

Conclusion

The class scheduler demonstrates how timer APIs support real-time browser interfaces and how ES modules divide responsibilities. Its countdown, scheduled reminder, and dynamically updated schedule show a practical pattern for building maintainable interactive applications.